

SQL for Data Analysis - Study Notes

Original study notes prepared for the DataMind RAG Assistant project.

What is SQL?

SQL (Structured Query Language) is the standard language used to define, query, and manipulate data stored in relational databases. A relational database organizes data into tables made up of rows and columns, where each row represents a single record and each column represents an attribute of that record.

Analysts use SQL to pull exactly the data they need out of large databases, filter and aggregate it, and combine information that is spread across multiple tables, before moving it into tools like Pandas, Excel, or a BI dashboard for further analysis.

SELECT, WHERE, and ORDER BY

The SELECT statement is the starting point of almost every SQL query. It specifies which columns to retrieve from a table, for example 'SELECT customer_id, order_date FROM orders'. Using SELECT * retrieves every column, which is convenient for exploration but often wasteful in production queries.

The WHERE clause filters rows based on a condition, such as 'WHERE order_date >= 2024-01-01'. Conditions can be combined with AND, OR, and NOT, and can use comparison operators, BETWEEN, IN, and LIKE for pattern matching on text.

ORDER BY sorts the result set by one or more columns, ascending by default or descending when the DESC keyword is added. LIMIT (or TOP, depending on the database engine) restricts how many rows are returned, which is useful when previewing large tables.

Aggregation: GROUP BY and aggregate functions

Aggregate functions such as COUNT, SUM, AVG, MIN, and MAX summarize many rows into a single value. On their own they summarize the whole table; combined with GROUP BY, they compute one summary value per group.

For example, 'SELECT region, SUM(revenue) FROM sales GROUP BY region' returns total revenue for each region. The HAVING clause filters groups after aggregation (for example, only regions with total revenue above a threshold), which is different from WHERE, which filters individual rows before aggregation.

What is a SQL JOIN?

A JOIN combines rows from two or more tables based on a related column between them, typically a key such as `customer_id`. Without joins, data that is normalized across multiple tables (customers, orders, products) could not be analyzed together in a single query.

An INNER JOIN returns only rows that have matching values in both tables. A LEFT JOIN returns all rows from the left table and the matching rows from the right table, filling in NULLs where there is no match. A RIGHT JOIN does the mirror image, and a FULL OUTER JOIN returns all rows from both tables, matched where possible.

Choosing the right join type matters for correctness: an INNER JOIN can silently drop rows that don't have a match, which is a common source of undercounting in analytical queries.

Common analytical patterns

Window functions (e.g. `ROW_NUMBER`, `RANK`, `SUM(...)` OVER (PARTITION BY ...)) let an analyst compute running totals, rankings, or per-group calculations without collapsing rows the way GROUP BY does, which is especially useful for time-series and leaderboard-style analysis.

Common Table Expressions (CTEs), written with the WITH keyword, let you break a complex query into named, readable steps, which is often preferred over deeply nested subqueries when preparing data for a downstream Pandas or visualization step.